Medical Imaging: Coursework Report

Author: Ilya Kogan
Data Intensive Science
DAMTP, University of Cambridge

April 4, 2025

Word Count: 2971

1 Introduction

This report explores different techniques across three areas of medical imaging: PET-CT image reconstruction, MRI image denoising, and CT image segmentation and classification. The primary objective of data science in medical imaging is to improve image quality and readability while automating some feature extraction, analysis, and interpretation. Our aim is to assist healthcare professionals in making more accurate and timely diagnoses, ultimately improving patient care and enhancing the quality of life.

2 Part 1: PET-CT Image Reconstruction

2.1 Understanding Data

After downloading and organizing data, we load in the data, using *np.load()* for each of the 5 files: CT sinogram, CT dark and flat fields, PET sinogram, and PET calibration data.

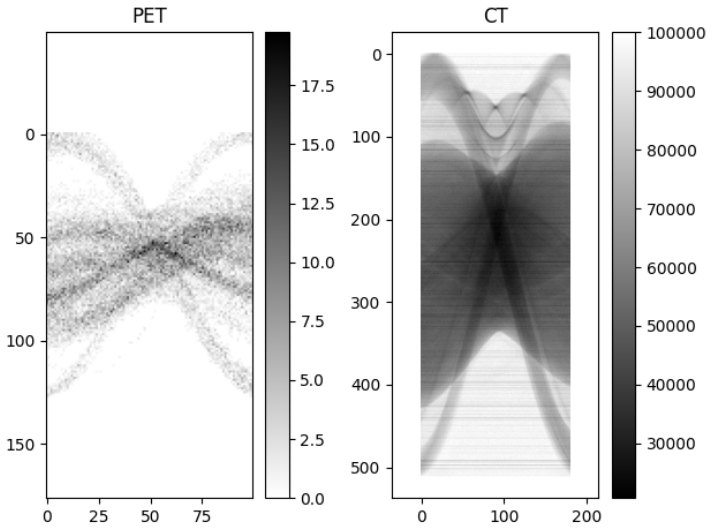

Figure 1: Displaying sinograms as measured/provided. PET sinogram is shown in reverse gray colorbar, as used by PET scientists. CT sinogram is shown in usual grayscale colors.

We note that the CT sinogram is a transmission sinogram since it is given to us as measured. We, also, notice that PET and CT images are of different sizes which means we will need to resize the CT image to obtain attenuation and correction data for our PET image.

2.2 Cleaning up sinograms

To clean the CT sinogram, we use provided fields. Dark fields are detector measurements when there is no X-ray and show our system noise (i.e. electronic or detector noise). Flat fields are measurements of uniform X-ray signal which, essentially, show the I_0 and can be used to normalize the image. These measurements are provided in *ct_dark.npy* and *ct_flat.npy*, respectively. Thus, to clean our CT sinogram of noise, we subtract the noise data from the sinogram and then divide by the cleaned flat field: *(ct_sino - ct_dark) / (ct_flat - ct_dark)*.

To clean the PET sinogram, we normalize it by the provided calibration data. Since we perform division, we check that we are not dividing by 0, or are rather correcting these NaN values to 0, using *np.where()* function.

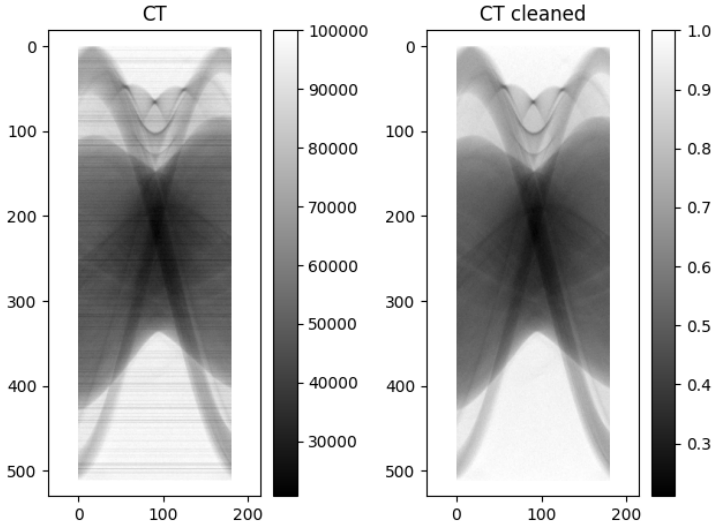

Figure 2: CT sinogram as provided vs cleaned. Even by the eye, we can see the horizontal lines cleaned out on the right image as compared to the initial image on the left. Moreover, we note the color scale is now normalized.

2.3 CT Reconstruction

To perform Filtered Back Projection (FBP) reconstruction for CT, shown in 4, we apply the inverse Radon transform, using *iradon*, on the cleaned absorption CT sinogram using measurements taken at 180 different angles over a total angle of 180°. The absorption sinogram is obtained from the clean sinogram by applying the negative logarithm, as per Beer Lambert Law.

To perform OS-SART, or Stochastic Gradient Descent (SGD), reconstruction for CT, shown in 3, we write a function, *os_sart_reconstruction*, which takes number of iterations, list of step sizes, number of subsets, and cleaned absorption sinogram as parameters and outputs the OS-SART reconstructed image. This function represents the equation of SGD, $x^{k+1} = x^k + \gamma \times A_i^T(A_i x^k - b)$,

with *num_parts=10* batches and 50 steps. For each step, we iterate over 10 batches, taking into account only a subset of angles and respective part of the sinogram, and update our current state, x_os_sart, one batch at a time. Thus, 1 step of the algorithm is 1 whole sinogram state update.

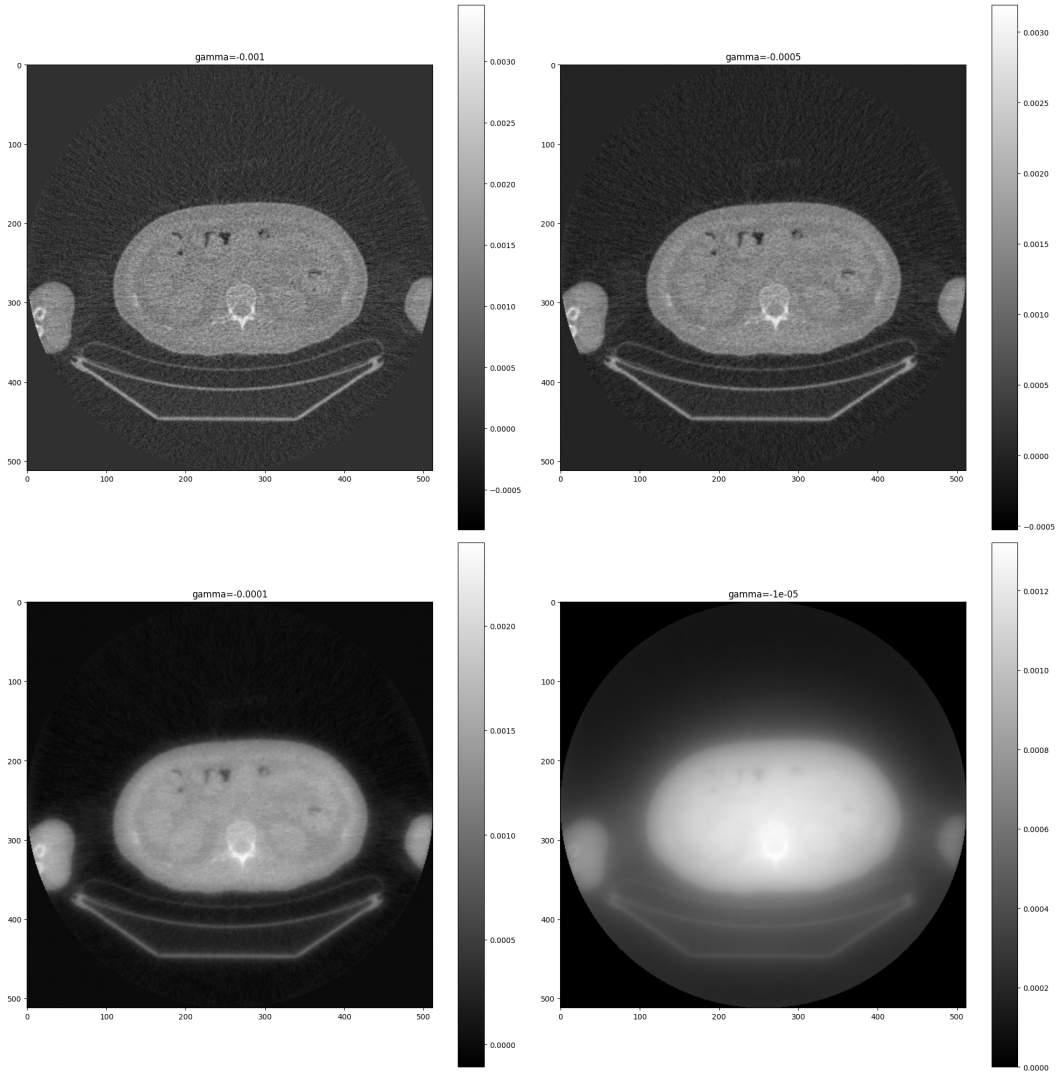

Figure 3: Illustrating the result of OS-SART image reconstruction performed with 4 different gammas, aka step sizes. The variation in gamma significantly affects image quality, noise levels, and the preservation of anatomical features.

We test different gammas which represent the step size and different number of steps to find values of γ and K that produces a better image than FBP,

some results shown in 3. Here, we define better as being less noisy which we can quantify using TV-norm (total variation norm), but at the same time not blurry (i.e. capturing all the information) which we can quantify using L1 and L2 norms. TV-norm is calculated as a sum of gradients and for FBP reconstructed image is equal to 128.73 (2 d.p.) while L1-norm is 0.42 (2 d.p.), and L2-norm is 0.33 (2 d.p.). We observe that $\gamma = -0.001$ recovers the image well visually and with K=50 has a TV-norm equal to 59.10 (2 d.p.), L1-norm equal to 0.37(2 d.p.), and L2-norm equal to 0.33 (2 d.p.). Visually, as shown in 4, it is already almost as good as the FBP reconstructed image, just a little blurrier. Setting an early stopping criteria in terms of the size of update made, we find that with $\gamma = -0.001$ and K=176, our OS-SART reconstruction outperforms the FBP reconstruction: L1 and L2 norms are close to each other while the TV-norm is 94.50 (2 d.p.) compared to 128.73 (2 d.p.) of the FBP reconstructed image.

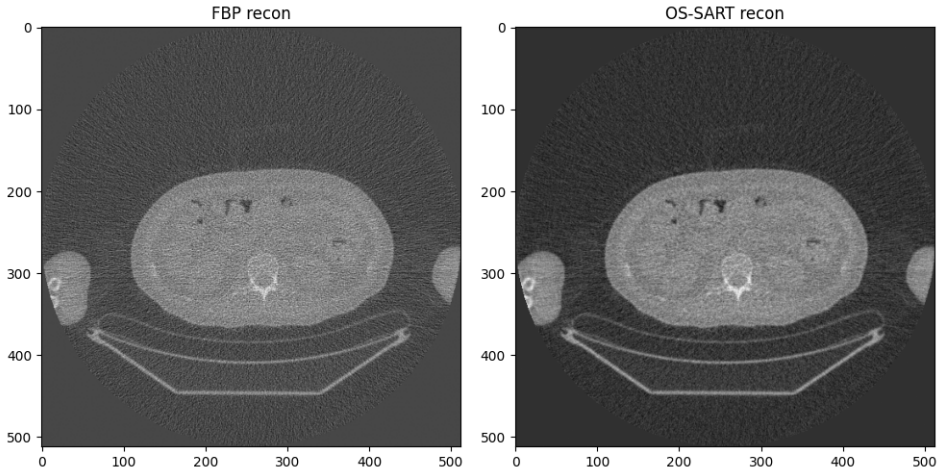

Figure 4: Showing a CT image that was reconstructed using FBP in comparison to the OS-SART reconstructed image. OS-SART reconstruction, obtained with parameters $\gamma = -0.001$ for step size and K=176 for number of steps, is less noisy with clearer defined features and is more visually appealing.

2.4 PET Attenuation Correction

To perform PET reconstruction we need to perform attenuation correction (scatter correction is omitted) using the CT image, but since CT and PET images are of different sizes, we need to resize the CT image first. Given that the PET image, at reconstruction, has pixels of 4.24 mm in size, and the CT image has pixels of of 1.06 mm, we have a scale factor equal to $4.24 \div 1.06 = 4$. We resize the FBP-reconstructed CT image, using *resize* function from skimage.transform. We are purposefully decreasing the quality of the CT image to match a coarser resolution of the PET image as shown in 5.

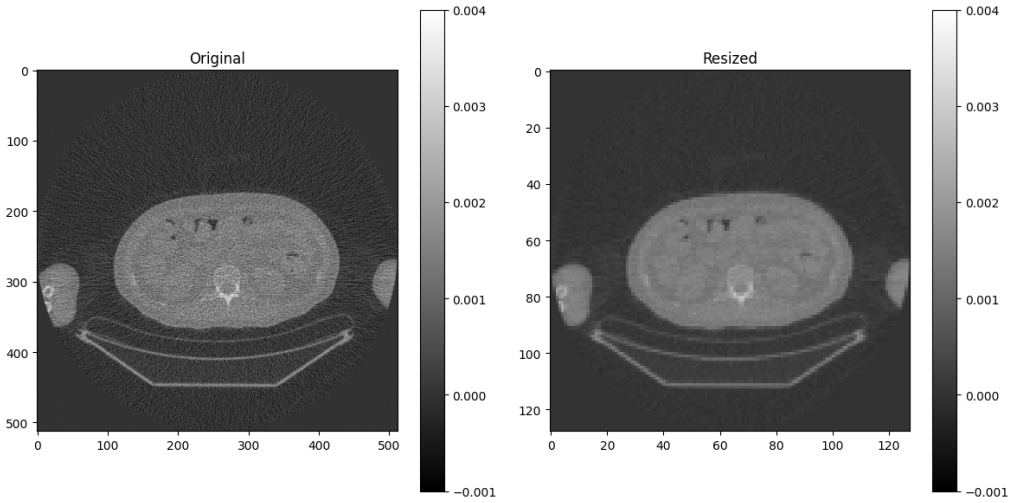

Figure 5: Comparison of original and resized CT images. On the left, is the original CT image in full resolution, while on the right is the resized CT image, adjusted to match the dimensions of the PET image. While the resized image retains most features from the original image, it loses some clarity. Nevertheless, this adjustment helps ensure accurate attenuation correction, which is our goal.

Using Radon transform implementation, *radon*, we then produce a sinogram of resized image with only 100 observations on total angle of 180° since this is the resolution of the provided PET image. This absorption sinogram is already clean since it was obtained from a resized image that was produced from the clean absorption sinogram. Now, we can correct for attenuation in PET using Beer Lambert Law: $I_{em} = I_{atte}/e^{-\sum \mu_i l_i}$ where $\sum \mu_i l_i$ come from the CT absorption sinogram.

2.5 PET Reconstruction

Finally, we can reconstruct our PET image. To do so using FBP, we apply *iradon* to the corrected for attenuation PET sinogram and get the image shown in 6.

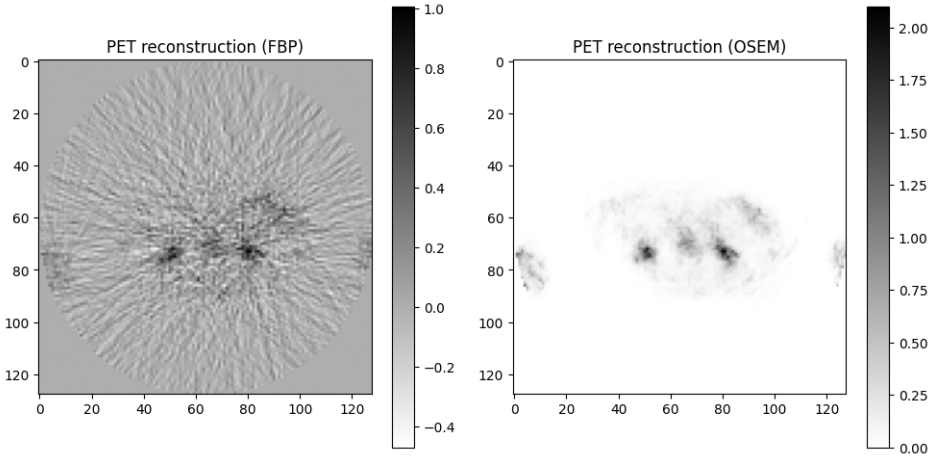

Figure 6: Comparison of PET reconstructions. On the left, is the PET image reconstructed using FBP; it has a lot of noise. On the right, is the PET image reconstructed using OSEM; it clearly shows areas of interest. The differences in reconstructed images showcases the effectiveness of OSEM. Both PET images are shown in reversed color as used by PET scientists.

To reconstruct our PET image using OSEM, an iterative method, we define the function, *os_sart_reconstruction*, which takes number of iterations, number of subsets, and emission data (corrected for attenuation sinogram) as parameters and outputs the OSEM reconstructed image. With values of $K \geq 10$, our OSEM reconstruction diverges as we don't have any step size regularization in the provided version of the algorithm: $x^{k+1} = x^k A_i^T(\frac{b}{A_i x^k})$. We find a less noisy image, in comparison to FBP reconstruction, as shown in 6 with a single iteration of the algorithm (k=1) when dividing data into 10 subsets.

2.6 Theoretical Q1: Overlaying PET and CT

Using *imshow* from matplotlib we overlay PET image over the resized CT image (since these are the images with same dimensions) to obtain the result shown in 7. The key step is to set the alpha, transparency parameter, of the top image to a suitable value, like 0.8. [1]

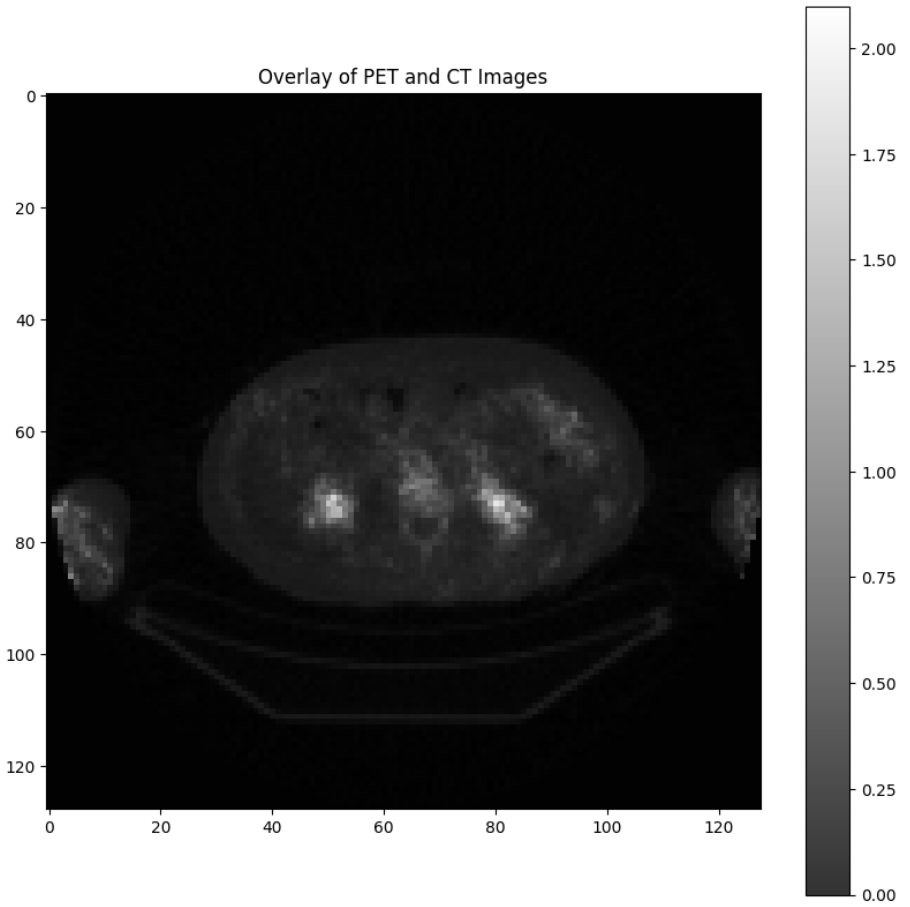

Figure 7: Displaying PET imaged overlaying the resized CT image. It highlights areas of metabolic activity and showcases anatomical structure. The gray scale represents intensity, identifying regions of interest.

2.7 Theoretical Q2: TOF-PET

If we had a TOF-PET (Time-of-Flight PET) scanner, we would need to measure time in nanoseconds to know the location of the emission so precisely that the reconstruction would not be needed. The signal that is collected during PET comes from annihilation of positron and an electron, leading to two photons, or X-rays, traveling in opposite directions, at 179.5° to be precise. With TOF-PET we want to derive the location of the source of these 2 X-rays, based on the time measurements of opposite detectors. Since we still need CT image for anatomical analysis, let's take the resolution of the given CT image, where 1 pixel is 1.06 mm in size. Thus, we want to be able to determine 1.06mm based on our time measuremnts of detectors. Given that speed of X-ray is equal to speed

of light which is 3×10^8 m/s, or 3×10^{11} mm/s. To be able to determine 1mm, we need time resolution of $\frac{1}{3\times10^{11}}$s, or 0.33×10^{-11}s, or 33ns (nanoseconds). Given error bounds, the actual measured time resultion should be twice as good, or 16ns.

2.8 Theoretical Q3: OSEM vs GD

OSEM is the most used algorithm in PET, but not in CT because PET reconstruction is more challenging as CT images are usually less noisy and using OSEM algorithm is, hence, unnecessary. Gradient Descent is an iterative algorithm that updates parameters of a model based on its cost (or loss) function, using its gradient. At each step, the algorithm moves in the direction of negative gradient, aiming to find global minima of the loss function. While OSEM is also an iterative algorithm, it is an Expectation-Maximization algorithm, working primarily with the likelihood function and not the loss function. Also, OSEM divides the data into subsets while in traditional GD algorithm the entire dataset is used at each iteration. However, there exist Stochastic, or batch, version of GD which uses subsets of data at each iteration.

2.9 Theoretical Q4: PET-MR

PET-MR combines two different imaging modalities while in a PET-CT image modality is the same, allowing for attenuation correction of PET using CT. Thus, the additional data processing step in PET-MR involves obtaining these attenuation coefficients. To enable accurate attenuation correction in PET-MR, attenuation maps are calculated using some calibration data that relates specific MRI signal intensities to known attenuation coefficients determined from CT data.

3 Part2: MRI Image Denoising

3.1 Visualisation and identifying noise

We load the complex data from *kspace.npy* using the *np.load* routine and find that our data shape is (6, 280, 280) which means that the 1st dimension is the coil dimension since there are 6 coils.

To create an image of the magnitude of k-space for each coil, we calculate the magnitude using *np.abs()* and then use matplotlib's *imshow()* function to obtain the images shown in 8.

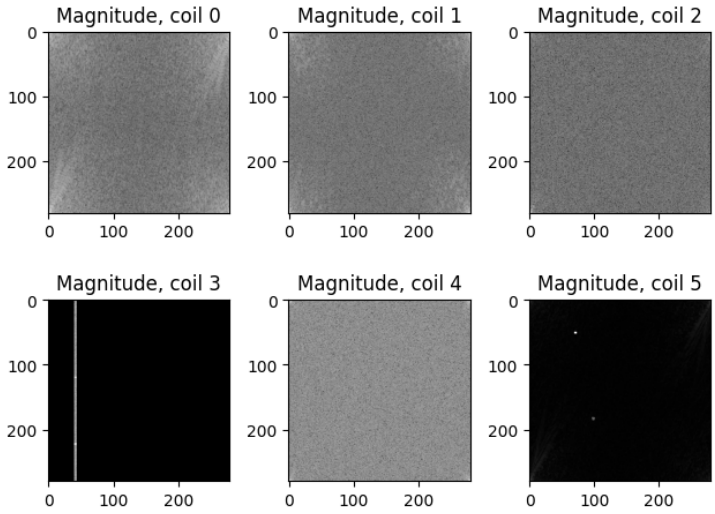

Figure 8: Magnitude images of k-space from 6 MRI coils. Each subplot represents the data collected by respective coil.

To transpose the data into the image space we apply the Fourier transform, using its numpy's implementation: *np.fft.ifft2*. To visualize the magnitude and phase from a coil, we use *np.abs()* and *np.angle()*, respectively. For the first coil, we get the visualization that is shown in 9. Magnitude images from all coils are shown in 10.

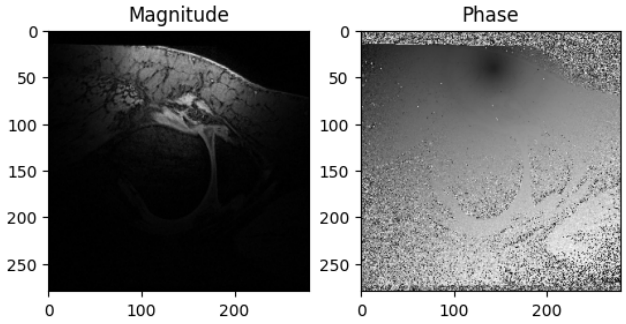

Figure 9: Displaying MRI magnitude and phase images from coil 1. The magnitude image shows tissue structure, while the phase image provides information about changes in magnetic field.

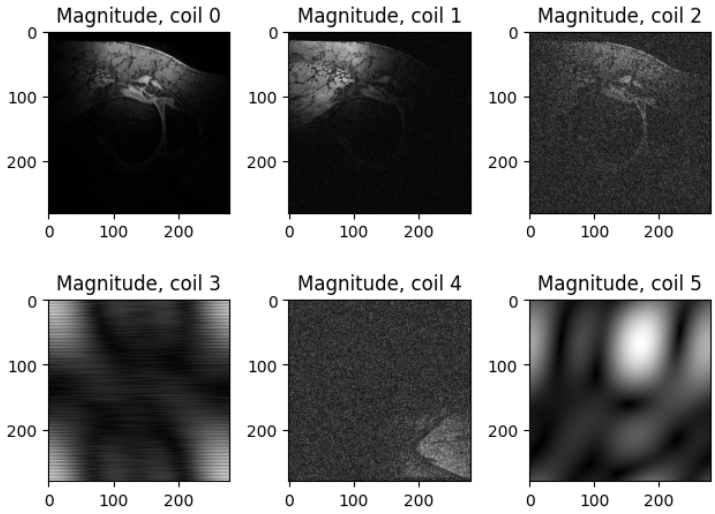

Figure 10: Magnitude images in spatial domain from 6 MRI coils. Each subplot represents the data collected by the respective coil. The differences in appearance reflect each coilindividual characteristic.

We combine images by squaring the image from each coil, summing the magnitudes, and then taking the square root of the final result. In the fully combined image, we observe that coils 0, 1, and 2 enhance each other and resulting image shows more structure and clarity while coils 3 and 5 intorduce noise. Effectively, we are overlaying the images from all coils on each other to obtain the combined image. We note that when combining all coils we get a noisy image, while combining coils 0, 1, 2, and 4 we get a much 'better' image as shown in 11.

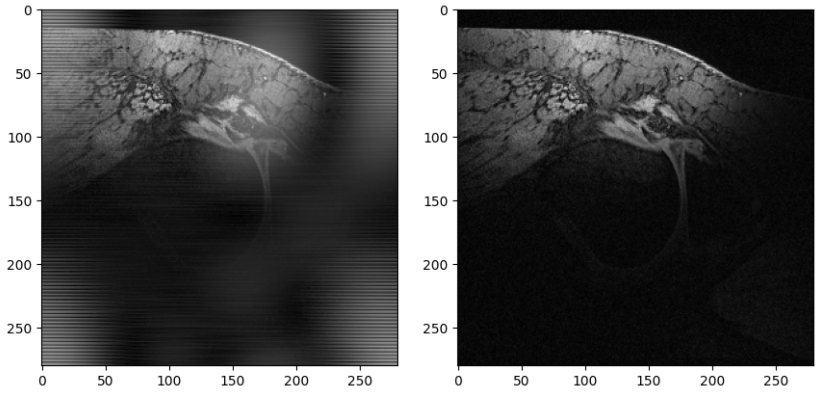

Figure 11: The left image was obtained by squaring the image from each coil, summing the magnitudes, and then taking the square root of the final result. The second image was obtained in the same way, but only using coils 0, 1, 2, and 4. This highlights the introduction of noise from coils 4 and 6.

3.2 Removing noise

Firstly, we apply a mean filter which smooths the image by taking an average over pixel values in a region, using *uniform_filter* from *scipy.ndimage*. This function involves a hyper-parameter that defines the size of the region over which the mean is calculated, the larger the size - the more blurred is the resulting image. As observed in 12, using a mean filter reduces noise but blurs the features as well.

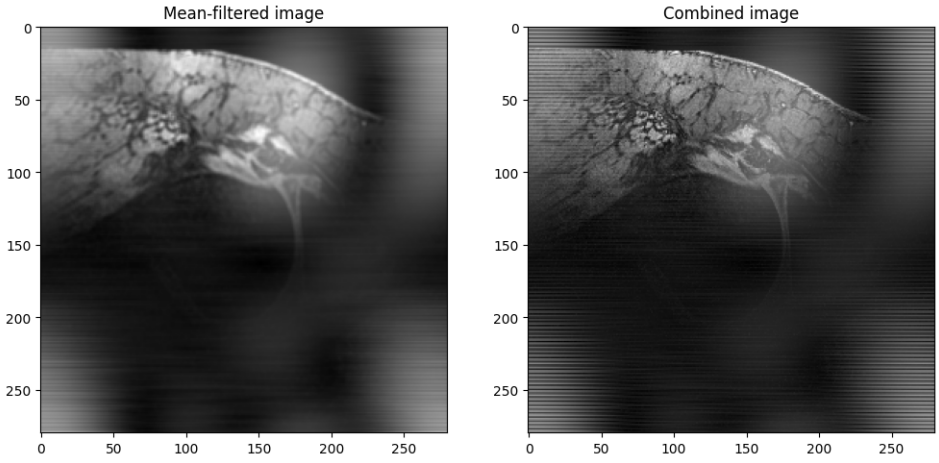

Figure 12: Displaying mean-filtered combined image (on the left) with kernel size 3. On the right, the image before mean-filtering is shown for comparison purposes.

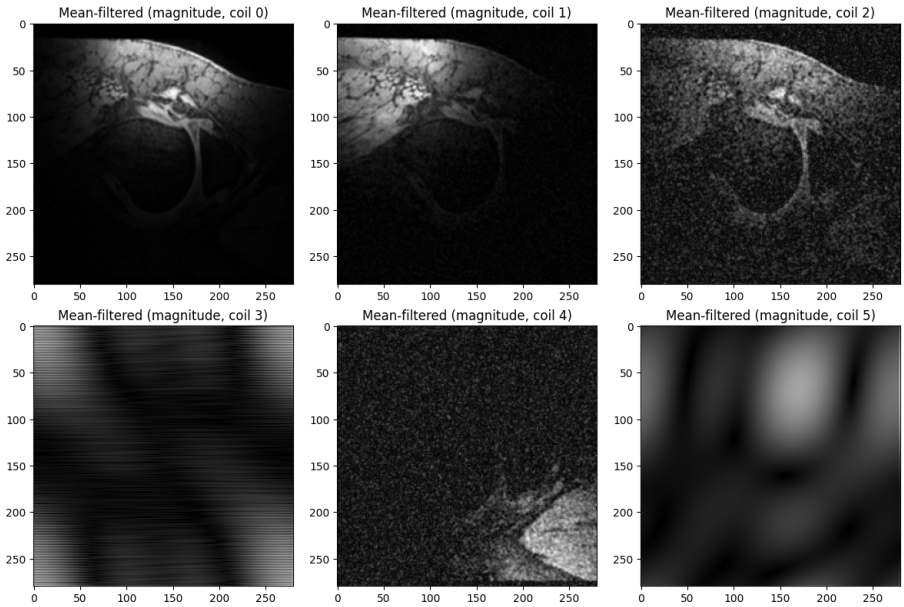

Figure 13: Displaying mean-filtered images for each coil. Each of the images from coils 0, 1, 2, and 4 show improvement with coils 2 and 4 improving the most. Images from coils 3 and 5 are just more blurred.

Secondly, we apply a Gaussian filter which acts similarly to the mean filter: it reduces noise while also smoothing the image. However, the Gaussian filter weights the pixel values with a Gaussian, meaning that closer pixels to the center of kernel get more representation. This results in slightly less blurring of features in comparison to the mean filter. We use the implementation from *scipy.ndimage* to obtain results shown in 14 with parameter describing the standard deviation of Gaussian along different axes equal to (0.8, 0.4). [2] We use different values for different axes since the most noise is present in horizontal form; such approach allows us to limit the loss of features while decreasing the amount of noise.

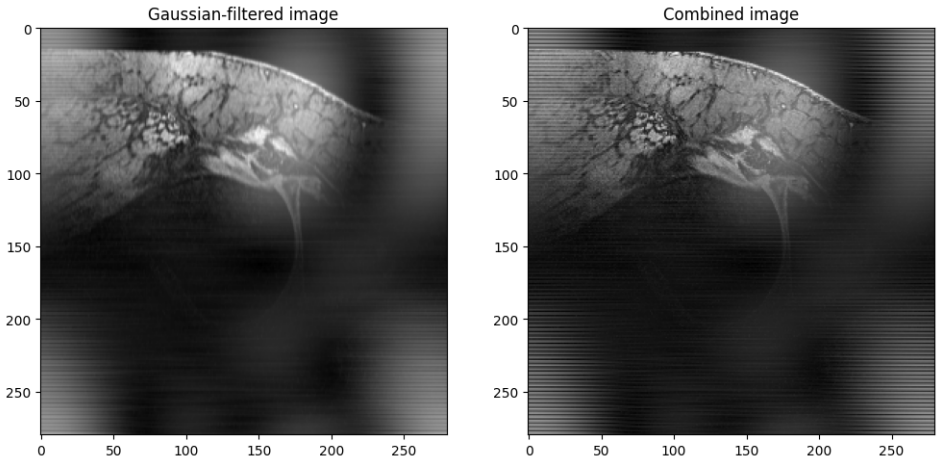

Figure 14: Displaying Gaussian-filtered image (on the left) with the standard deviation of Gaussian along different axes equal to (0.8, 0.4). On the right, the image before filtering is shown for comparison purposes.

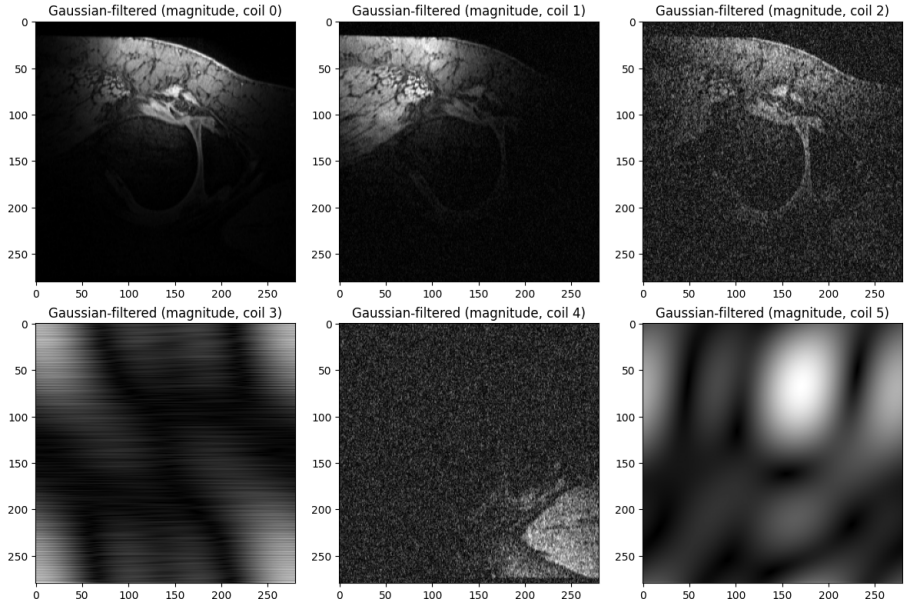

Figure 15: Displaying Gaussian-filtered images from each coil due to confusing instructions. As we can see, these images retain more features and are more sharp than mean-filtered images. Even the noisiest images, from coil 3 and 5, show more refined structure.

Thirdly, we apply a wavelet filter in its *skimage.restoration* implementation,

specifically the one using BayesShrink algorithm. It is an adaptive approach with wavelet soft thresholding where a unique threshold is estimated for each wavelet subband which generally results in an improvement over what can be obtained with a single threshold.[3] This denoising technique is quite powerful as it further improves the preservation of edges and details of the image in comparison to the mean-filter as shown in 17.

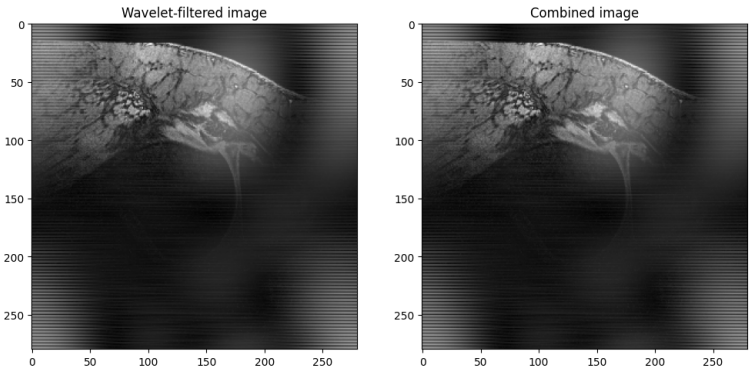

Figure 16: Displaying Wavelet-filtered image (on the left) obtained with BayesShrink. On the right, the image before filtering is shown for comparison purposes.

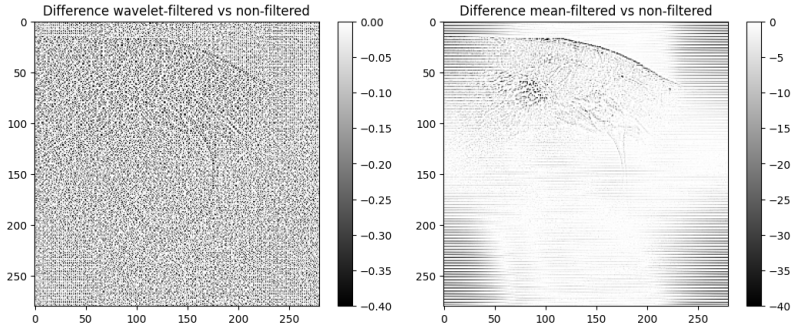

Figure 17: Displaying the difference between Wavelet-filtered image obtained with BayesShrink and the mean-filtered combined image. Illustrates higher preservation of edges and features in wavelet-filtered images.

Additionally, we explored a few other filters. Here we just present one more, a vertical Farid transform filter. The purpose of this filter is to find the vertical edges of an image using the Farid transform [4] which seems useful in our case, given a lot of horizontally aligned noise. Indeed, the resulting image, as shown in 18, provides a lot of structure/texture information for the regions we are interested in while completely eliminating annoying horizontal lines. It does lose some horizontal features which is expected to happen, but the overall result of applying this filter is impressive.

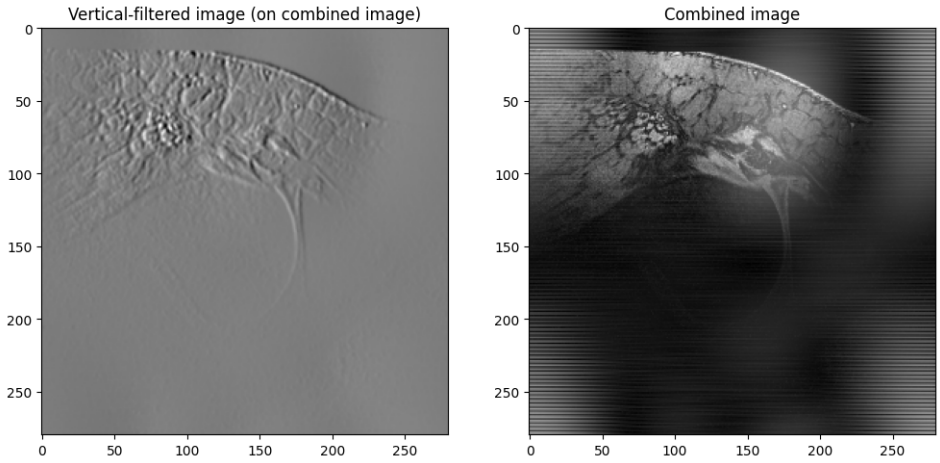

Figure 18: Displaying the result of filtering by Farid transform in comparison to the original combined image. Illustrates the importance of unserstanding noise type and form: understanding what kind of noise is present in the image facilitates the denoising process.

Overall, it is really important to know which type of noise we are expecting to correct for. This information would promote the specific filter usage which would improve the image the most.

3.3 Butterworth filter

We define the low-pass Butterworth filter as provided. By applying it with default parameters, we find that the cutoff frequency is too low as almost all information is lost. By experimenting, we choose the cutoff frequency of 250 (for n=2) to obtain result shown in 19.

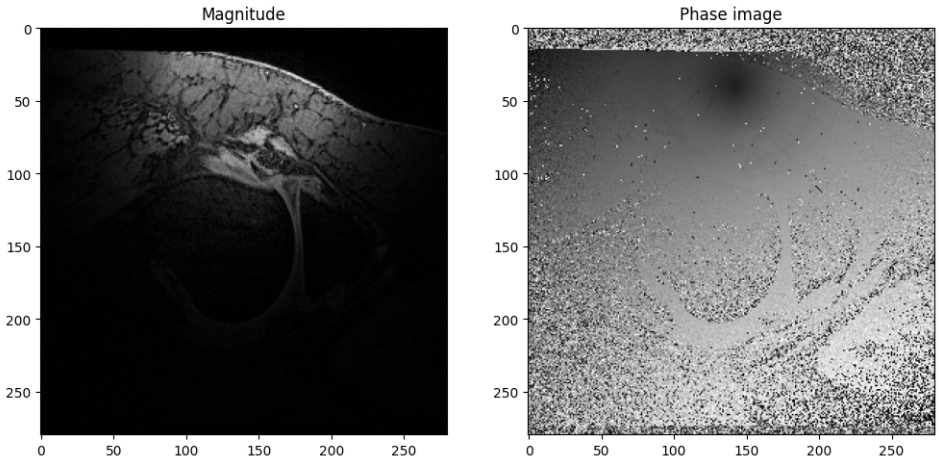

Figure 19: Displaying the result of filtering using low-pass Butterworth filter with cutoff frequency of 300 (n=2)

Butterworth Filter, or denoising in k-space in general, changes the image in the frequency domain which leads to more precise results. For example, if the frequency of noise is known, it can be removed very efficiently without affecting signal at any other frequency (i.e. using a band-pass filter), leading to better edge and feature preservation. Image-based methods, on the other hand, operate on the image and usually are not as direct or direct. From examples seen above, they often involve blurring and issues with edge preservation. Moreover, k-space filtering might be more efficient for large datasets than image-based filtering.

To recreate a new combined image, we, first, apply the Wavelet filter to all coil images and then combine them by squaring the image from each coil, summing the magnitudes, and taking the square root to obtain the image shown in 20.

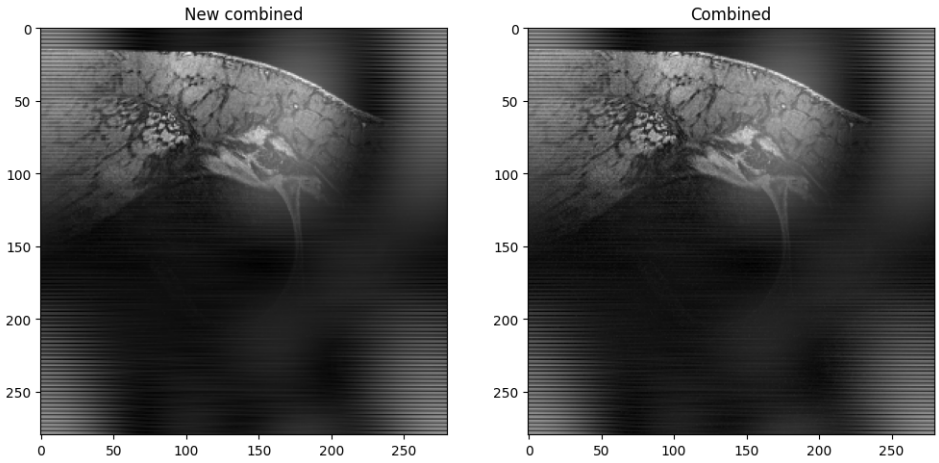

Figure 20: Displaying the result of combining images after applying the Wavelet filter on the left and combined image from unfiltered coil images on the right. Even though difference is hardly noticable by eye, the newly combined image does encompass features slightly better.

Methods that might be able to further improve the combined image quality include Machine Learning frameworks, like Deep Learning, which can learn how to get from noisy to noiseless images through training. Furthermore, iterative denoising methods can be used, filtering image with various techniques at different steps of denoising process. Additionally, generative models can be used as denoisers.

4 Part3: CT Image segmentation and classification

4.1 Image segmentation

To open the NIfTI files and create numpy arrays, one per patient scan and one per segmentation mask, we use SimpleITK library and its documentation, [5]. To read in the images, we use: *ReadImage()* method and to obtain arrays from them, we use *GetArrayViewFromImage()*. We iteratively read in all images, get their array representations and apply the segmentation masks to extract specific regions of interest within the 3D image.

We, also, create our own segmentation mask using a simple thresholding algorithm. Our method is working as expected: it successfully masks some of the image, but not all of it. Since these are medical images, in 3D, we can expect to see other features at other slices; therefore, we can not expect to perform an accurate segmentation solely by thresholding based on values in the area of interest. Overall, our accuracy match is 0.10 (2 d.p.) but that is given that average size of interest region to mask size is: 0.03 (2 d.p.). They match in

region of interest and even in other areas, but the size of interest region is too small compared to the size of the subvolumes to provide meaningful results on the whole input.

There exist numerous ways to improve our custom segmentation process. Firstly, we can tighten the threshold boundaries if we want to observe only specific features. Secondly, we can use an Otsu algorithm which is based on finding the optimal separation value between background and object, based on image histogram. Furthermore, there exist numerous ML approaches, such as SVMs, k-means clustering and more.

4.2 Image feature extraction and classification

We define functions to calculate three histogram-based radiomic features on the segmented regions of each image: Energy - a measurement of the intensity of voxels, $E = \sum_{i=1}^{N}(V_i)^2$; MAD - the mean difference between intensity values and the Mean Value over region of interest (ROI), $MAD = \frac{1}{N}\sum_{i=1}^{N}|V_i - \bar{V}|$; and Uniformity - a measurement of the homogeneity of the image, $U = \sum_{i=1}^{M} p_i^2$. Uniformity values range from 0 to 1: with 1 indicating perfect homogeneity (all intensities are in one bin), and 0 indicating complete heterogeneity (intensities are evenly spread across bins). To calculate uniformity, we, first, estimate the number of bins that would be suitable for all images by calculating a square root of the mean number of voxels in ROIs.

After calculating the energy, MAD, and uniformity metrics for each ROI, we construct a dataframe with each row representing a case (patient). This data format and organization will facilitate the calculation of relevance of each metric for the classification of diagnoses in our patients. Using the constructed dataframe, we compute the correlation matrix with *.corr()* method, shown in 21. As shown in the first row, each metric is positively correlated with the diagnosis, which means that each metric is relevant and helpful in determining/classifying the correct diagnosis, with energy metric carrying the highest amount of information, and MAD metric - the least; at least, in our dataset. [6]

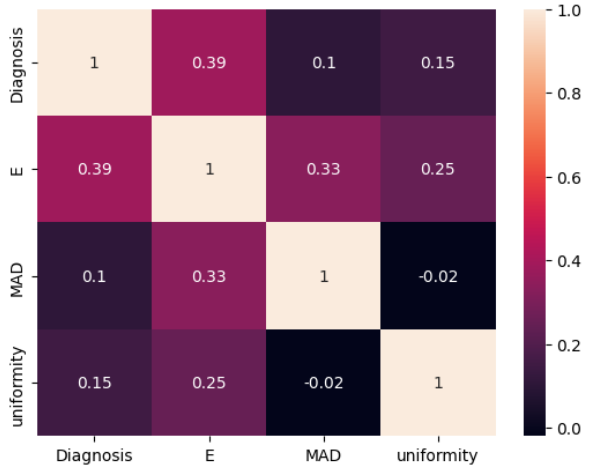

Figure 21: Displaying the correlation matrix between our metrics and diagnosis. Energy is the most correlated feature to the diagnosis, carrying a lot of information; for instance, high energy values may indicate areas with high activity for malignant lesions.

We, also, estimate the relevance of each feature by building a Random Forest Classifier and fitting it to our data. By extracting the model features importance, we find that it is 0.45 (2 d.p.) for E, 0.28 (2 d.p.) for uniformity, and 0.27 (2 d.p.) for MAD - similar results as from our covariance matrix calculations.

We conclude that using all three features is the best approach that would lead to the best classification between benign and malignant lesions in our dataset and, in general. Malignant lesions should have higher energy (more activity), higher MAD (more irregularity), and uniformity values closer to 1 as they would exhibit additional tissues which are usually more active than regular body tissue. However, these should not be regarded as general rule (perhaps, except for energy) and should be based on medical information.

5 Summary

In this report, we present our process and results of PET and CT image reconstructions, MRI denoising, and CT image segmentation and classification. We implemented steps like sinogram cleaning, PET attenuation correction, reconstruction using FBP in comparison to an iterative algorithm, OSEM. We, also, applied multiple denoising filters to MRi data comparing and contrasting their effects. Moreover, we created a basic segmentation mask and compared it to the truth one, gaining insight into segmentation process. Furthermore, we examined different metrics, such as energy, MAD, and uniformity and their importance in tumor classification, evaluated on our data. Everything was implemented using best software practices. Overall, this report demonstrates the potential for ML techniques to significantly aid clinical diagnostics.

References

[1] Matplotlib Development Team. *Layer Images*. Accessed: 2025-04-01. 2023. URL: https://matplotlib.org/stable/gallery/images_contours_and_fields/layer_images.html.

[2] Pauli Virtanen et al. *scipy.ndimage.gaussian$_f$ilter*. Accessed: 2025-02-04. 2023. URL: https://docs.scipy.org/doc/scipy/reference/generated/scipy.ndimage.gaussian_filter.html.

[3] Scikit-image developers. *Denoising using wavelets*. Accessed: 2025-02-04. 2023. URL: https://scikit-image.org/docs/0.24.x/auto_examples/filters/plot_denoise_wavelet.html.

[4] scikit-image developers. *skimage.filters.farid$_v$*. Accessed: 2025-02-04. 2023. URL: https://scikit-image.org/docs/0.25.x/api/skimage.filters.html#skimage.filters.farid_v.

[5] Insight Software Consortium. *SimpleITK Notebooks*. https://insightsoftwareconsortium.github.io/SimpleITK-Notebooks/. Accessed: 2025-04-03. 2023.

[6] S. G. Armato III et al. *Data From LIDC-IDRI*. Data set. 2015. URL: https://doi.org/10.7937/K9/TCIA.2015.LO9QL9SX.

A Appendix

ChatGPT was used to help with formatting of this LATEX document, specifically with the following tasks:

- inserting and formatting images
- creating bibtex citations from sources